

ASSIGNMENT 3 — CODING & SIMULATION-BASED MICRO-PROJECT

CAPP and AS/RS Storage Optimization Algorithms

Test Verification Report and Code Repository · execution logs across at least three numerical test cases per task

Particulars	Details
Name of the student	
Enrolment / Roll number · Class	
Subject	Automation in Manufacturing · PBL Assignment 3
Target Course Outcomes	CO-3, CO-4
Date of submission	

1. Scope and Execution

Five standalone Python scripts, one per task, each fully commented and each ending with a `main()` that runs **at least three distinct numerical test cases** and prints an execution log. Every script also asserts its own results, so a script producing a wrong answer fails rather than printing it. All five were executed and **all five exited with status 0**.

```
cd Assignment3_Coding
python3 task3_1_capp_feature_recognizer.py
python3 task3_2_tool_offset_compensation.py
python3 task3_3_asrs_cycle_time.py
python3 task3_4_agv_fleet_sizing.py
python3 task3_5_barcode_verification.py

Standard library only - no third-party packages required.
```

Task	Script (listing in appendix)	Test cases	Exit
3.1 Generative CAPP feature recognizer	task3_1_capp_feature_recognizer.py — App. A	3	0
3.2 Tool offset / cutter radius compensation	task3_2_tool_offset_compensation.py — App. B	3	0
3.3 AS/RS single- and dual-command cycle time	task3_3_asrs_cycle_time.py — App. C	3	0
3.4 Material handling fleet sizing	task3_4_agv_fleet_sizing.py — App. D	3 + 7-point sweep	0
3.5 ADC barcode verification	task3_5_barcode_verification.py — App. E	11 scans	0

2. Task 3.1 — Generative CAPP Feature Recognizer

Accepts the part feature parameters — stock diameter, finished turned steps, chamfer angle, thread pitch, groove size — and outputs the ordered machining sequence, the

selected tools, cutting speeds, feeds, depths of cut, pass counts and estimated machining time. Features are classified, ordered by a precedence table, costed from a cutting-data table indexed by (material, operation), with $N = 1000 \cdot V_c / (\pi \cdot D)$ and $t = L / (f \cdot N)$.

	Case 1	Case 2	Case 3
Component	Stepped transmission shaft	Aluminium locating pin	Stainless valve stem
Material	Mild steel	Aluminium	Stainless steel
Stock	Ø50 × 100 mm	Ø25 × 60 mm	Ø90 × 150 mm
Steps	Ø30×25, Ø40×30, Ø50×40	Ø12×20, Ø20×25	Ø48×40, Ø70×45, Ø90×50
Thread	M30 × 1.5, 20 mm	none — tests the skip branch	M48 × 2.5, 35 mm
Groove	4 × 2 mm	none	6 × 3 mm
Operations generated	9	7	9
Total cycle time	2.303 min	0.718 min	20.198 min

Execution log — Test Case 1:

# Operation	Tool	Vc	N	f	ap	x	t,min
1 Face the bar end	T0101	180	1146	0.200	1.00	1	0.189
2 Rough turn step 1 to Ø30.0	T0101	200	1592	0.300	2.50	4	0.289
3 Rough turn step 2 to Ø40.0	T0101	200	1415	0.300	2.50	2	0.339
4 Thread relief groove	T0303	120	1273	0.080	2.00	2	0.119
5 Finish turn step 1 to Ø30.0	T0202	260	2759	0.120	0.40	1	0.156
6 Finish turn step 2 to Ø40.0	T0202	260	2069	0.120	0.40	1	0.302
7 Chamfer 2.0×45°	T0202	220	2334	0.100	0.50	1	0.092
8 Thread M30×1.5 over 20.0 mm	T0404	100	1061	1.500	-	9	0.227
9 Part off component	T0303	110	700	0.070	2.50	1	0.590
TOTAL CYCLE TIME (machining + approach)							2.303
>>> SANITY CHECKS PASSED for this case							

Self-checks asserted: facing is first and part-off last; the sequence is in precedence order; every spindle speed is positive and within the machine limit; the threading feed equals the pitch; total time is positive.

3. Task 3.2 — Tool Offset and Cutter Radius Compensation

Given a part profile and a tool nose radius or cutter radius with an orientation vector, computes the corrected tool-path coordinates. One geometry engine serves both cases: every straight segment is offset perpendicular to itself by r and neighbouring offset lines are intersected to form a mitre joint; circular contours are chorded into polylines.

	Case 1 — turning	Case 2 — milling, inside	Case 3 — milling, outside + circular
Geometry	Ø30 cylinder → 26.57° taper → Ø50 cylinder	Rectangular pocket 60 × 40 mm	Five-sided outside contour; Ø40 circular pocket
Tool	R = 0.8 mm, tip vector (−0.707, −0.707)	Ø10 cutter, G41	Ø12 cutter G42; Ø10 cutter G41

	Case 1 — turning	Case 2 — milling, inside	Case 3 — milling, outside + circular
Quantity checked	Profile error if compensation omitted	Cutter-centre rectangle	Clearance from profile; cutter-centre radius
Computed	0.2343 mm undercut on the taper	50.0000 × 30.0000 mm	14.9809 mm
Expected	> 0 (a taper must produce an error)	$(W-D) \times (H-D) = 50.0000 \times 30.0000$ — exact	$(40-10)/2 = 15.0000$ mm
Verdict	PASS	PASS	PASS

Case 1 shows **why** compensation matters: on cylindrical and facing cuts the imaginary tool tip traces the profile exactly, so no error arises; only on the **taper** does the contact point move round the nose radius, giving 0.2343 mm — enough to scrap the part.

On the 0.019 mm in Case 3 — the Ø40 pocket is chorded into 36 segments, so the path follows the inscribed polygon rather than the true circle. 14.9809 against 15.0000 is exactly that chord error, and it falls as the segment count rises. It is a property of the discretisation, not of the compensation.

Self-checks asserted: the nose centre never comes closer than R to the finished profile (no gouge); a taper must produce a measurable uncompensated error; the inside offset of a rectangle shrinks it by exactly one cutter diameter and starts exactly one radius from every wall; every cutter-centre point on the outside contour is at least one radius clear.

4. Task 3.3 — AS/RS Single- and Dual-Command Cycle Time

For a single-rack AS/RS, computes SC and DC cycle times and throughput for **randomized** and **dedicated** storage, from rack length and height, horizontal and vertical velocities and the P/D time. Both axes drive simultaneously, so travel is the Chebyshev time $\max(\Delta y/V_y, \Delta z/V_z)$.

```

th = L / Vy      tv = H / Vz
T = max(th, tv)  scaling factor
b = min(th, tv) / T  rack shape factor, 0 < b <= 1

randomized storage (Bozer & White):
E(SC) = T ( 1 + b^2 / 3 )      SC cycle = E(SC) + 2 tpd -> 1 transaction
E(DC) = T ( 4/3 + b^2/2 - b^3/30 ) DC cycle = E(DC) + 4 tpd -> 2 transactions

```

	Case 1 — tall rack	Case 2 — long rack	Case 3 — square in time
Rack L × H	60 m × 18 m	90 m × 6 m	40 m × 10 m
Vy / Vz	2.0 / 0.5 m/s	1.2 / 0.8 m/s	2.0 / 0.5 m/s
t _{pd} , availability	15 s, 95 %	8 s, 90 %	10 s, 100 %
th / tv	30.0 / 36.0 s	75.0 / 7.5 s	20.0 / 20.0 s
Shape factor b	0.8333	0.1000	1.0000
SC cycle	74.33 s	91.25 s	46.67 s

	Case 1 — tall rack	Case 2 — long rack	Case 3 — square in time
DC cycle	119.81 s	132.37 s	76.00 s
DC throughput	57.1 /h	49.0 /h	94.7 /h
Monte-Carlo deviation, SC	0.05 %	0.19 %	0.09 %
Monte-Carlo deviation, DC	0.10 %	0.09 %	0.09 %

Independent verification — the analytical formulae are not simply trusted. The script re-derives E(SC) and E(DC) by **Monte-Carlo simulation over 400 000 randomly drawn storage locations** and asserts agreement to better than 1 %. The measured deviations are 0.05 % to 0.19 %, so the implementation and the theory agree.

The three racks span the whole range of the shape factor — vertical-dominant, strongly horizontal-dominant ($b = 0.1$), and square in time ($b = 1$), the limiting case. For $b = 1$ the closed-form limits are also checked analytically:

$$\begin{aligned}
 b = 1 &\Rightarrow E(SC) = T \times 4/3 &= 20 \times 1.3333 = 26.667 \text{ s} &\text{OK} \\
 b = 1 &\Rightarrow E(DC) = T \times (4/3 + 1/2 - 1/30) &= 20 \times 1.8000 = 36.000 \text{ s} &\text{OK}
 \end{aligned}$$

In every case the DC cycle takes longer but gives **higher throughput**, because it completes two transactions per cycle — the result that justifies dual-command operation. The gain is largest for the square-in-time rack (+22.8 %).

5. Task 3.4 — Material Handling Fleet Sizing

Computes the AGV fleet size from loading and unloading times, loaded and empty travel distances, vehicle velocity, traffic delay factor and required deliveries per hour. If the traffic factor is **not supplied** the calculation is implicit — F_t depends on the fleet size and the fleet size on F_t — and is solved by fixed-point iteration.

$$\begin{aligned}
 T_c &= T_L + L_d/V_c + T_U + L_e/V_c & W_L &= R_f \times T_c \\
 A_T &= 60 \times A \times F_t & n &= W_L / A_T, \text{ rounded UP}
 \end{aligned}$$

	Case 1 — FMC loop	Case 2 — congested feed	Case 3 — F_t estimated
TL / TU	0.60 / 0.50 min	1.00 / 0.75 min	0.75 / 0.75 min
Ld / Le	140 / 110 m	320 / 280 m	200 / 150 m
Vc	50 m/min	40 m/min	45 m/min
Demand R_f	22 /h	35 /h	40 /h
Availability / traffic factor	0.95 / 0.90	0.90 / 0.85	0.92 / not given
Cycle time T_c	6.100 min	16.750 min	9.278 min
Workload W_L	134.20 min/h	586.25 min/h	371.11 min/h

	Case 1 — FMC loop	Case 2 — congested feed	Case 3 — Ft estimated
Vehicles (exact)	2.616	12.772	11.205
Fleet size	3 AGVs	13 AGVs	12 AGVs
Utilisation	87.2 %	98.2 %	93.4 %
Capacity vs demand	25.23 vs 22.00 /h OK	35.62 vs 35.00 /h OK	42.84 vs 40.00 /h OK

Case 3 is the implicit one: with no traffic factor given, the iteration converged in **3 steps** to $Ft = 0.600$, giving 12 AGVs. Case 1 is the loop designed in Assignment 1, and its result — 3 AGVs at 87.2 % — is the figure quoted there.

Self-checks asserted: cycle time and workload match the hand formulae to 10^{-9} ; the fleet size is rounded up, and by less than one vehicle; the chosen fleet can actually serve the demand; utilisation lies in $(0, 1]$. A demand sweep confirms the fleet size rises **monotonically** and never falls:

Demand, deliveries/h	n exact	AGVs required
5	0.595	1
10	1.189	2
20	2.378	3
30	3.567	4
40	4.756	5
60	7.135	8
80	9.513	10
>>> fleet size rises monotonically with demand		

6. Task 3.5 — ADC Barcode Verification Tool

Parses raw scanned barcode strings, extracts the part serial number, operation routing code and batch data, and validates the scan against a sample part database so that a component can only be released to the operation its routing permits.

CODE 39	fixed-field, * start/stop sentinel, modulo-43 check character: *<PART:8><OP:3><LOT:6><SERIAL:6><CHECK:1>* e.g. *BLKA12340102601150001238*
QR / 2-D	pipe-delimited key=value, any field order: PN=BLK-A1234 OP=010 LOT=260115 SN=000123

Checks performed: sentinel and field format; the Code 39 modulo-43 check character; the part number exists in the master; the operation belongs to that part's routing; the operation is the **correct next** one for that serial (sequence enforced, not just membership); the lot is a real date, not in the future and inside shelf life; and the serial has not already been scanned at that operation.

#	Scan	Result	Expected	Verdict
1	Valid Code 39, operation 010 (first in routing)	ACCEPT	ACCEPT	OK
2	Valid Code 39, operation 020 (correct next)	ACCEPT	ACCEPT	OK
3	QR, operation 060 presented while 030 is due	REJECT	REJECT	OK
4	QR, operation 020 presented a second time	REJECT	REJECT	OK

#	Scan	Result	Expected	Verdict
5	QR, valid first operation on a different part	ACCEPT	ACCEPT	OK
6	Code 39 with one bit flipped in the check character	REJECT	REJECT	OK
7	QR, part number not in the master	REJECT	REJECT	OK
8	QR, lot dated in the future	REJECT	REJECT	OK
9	QR, lot older than the 180-day shelf life	REJECT	REJECT	OK
10	Unreadable payload ("HELLO WORLD 12345")	REJECT	REJECT	OK
11	Truncated Code 39 payload ("*SHORT*")	REJECT	REJECT	OK

3 accepted, 8 rejected, 11 of 11 verdicts as expected. The validation date is fixed at 1 March 2026 inside the script so the shelf-life and future-date tests give the same answer whenever it is run. The routing history built during the run shows state is tracked correctly across scans:

```
Routing history recorded by the validator:
BLK-A1234/260115/000123      operations completed: 010-020
SHF-T5500/260210/004501      operations completed: 010
```

7. Summary

Task	Test cases	Verification method	Result
3.1 Generative CAPP	3 components, 3 materials	6 assertions per case	PASS
3.2 Tool / cutter compensation	3 geometries, turning and milling	6 assertions including exact-value checks	PASS
3.3 AS/RS cycle time	3 rack shapes, $b = 0.1$ to 1.0	Monte-Carlo cross-check over 400 000 trials + analytical limits at $b = 1$	PASS
3.4 AGV fleet sizing	3 systems + 7-point sweep	6 assertions + monotonicity check	PASS
3.5 Barcode verification	11 scans: 3 valid, 8 faulty	Every verdict asserted against its expected value	PASS

Why the results are meaningful — the scripts assert properties that would fail if the implementation were wrong: exact geometric identities in Task 3.2, an independent Monte-Carlo derivation in Task 3.3, a monotonicity sweep in Task 3.4, and deliberately faulty inputs that **must** be rejected in Task 3.5. Three of the five tasks therefore carry a negative control.

```
#!/usr/bin/env python3
"""
Task 3.1 - Generative CAPP Feature Recognizer
=====
Accepts the feature parameters of a turned component (stock diameter, the final
turned steps, chamfer angle, thread pitch) and automatically generates:

    * the ordered machining sequence,
    * the cutting tool selected for every operation,
    * cutting speed (Vc), spindle speed (N) and feed (f),
    * depth of cut and the number of passes,
    * the estimated machining time for each operation and for the whole part.

Method
-----
1. FEATURE RECOGNITION - the input geometry is scanned and classified into
    machinable features (face, cylindrical step, chamfer, groove, thread,
    parting face).
2. OPERATION SEQUENCING - features are ordered by the standard turning
    precedence rules held in PRECEDENCE (face first, roughing before
    finishing, thread after its relief groove, part-off last).
3. TOOL + CUTTING DATA SELECTION - a tool is chosen per operation from
    TOOL LIBRARY and the cutting data is looked up from CUTTING_DATA, which is
    indexed by (work material, operation class).
4. PROCESS CALCULATION -
    N = 1000 * Vc / (pi * D)          [rev/min]
    t = L / (f * N) per pass          [min]
    passes_rough = ceil(radial stock / ap)

Run: python3 task3_1_capp_feature_recognizer.py
"""

from __future__ import annotations

import math
from dataclasses import dataclass, field
from typing import List, Dict, Optional

# -----
# 1. KNOWLEDGE BASE
# -----

# Cutting data: {material: {operation class: (Vc m/min, f mm/rev, ap mm)}}
# Values are representative of uncoated/coated carbide tooling on a rigid CNC
# turning centre and are deliberately conservative.
CUTTING_DATA: Dict[str, Dict[str, tuple]] = {
    "MILD_STEEL": {
        "FACE": (180, 0.20, 1.0), "ROUGH": (200, 0.30, 2.5),
        "FINISH": (260, 0.12, 0.4), "GROOVE": (120, 0.08, 2.0),
        "THREAD": (100, None, None), "PART_OFF": (110, 0.07, 2.5),
        "CHAMFER": (220, 0.10, 0.5)
    },
    "ALLOY_STEEL": {
        "FACE": (140, 0.18, 1.0), "ROUGH": (160, 0.25, 2.0),
        "FINISH": (200, 0.10, 0.3), "GROOVE": (90, 0.06, 2.0),
        "THREAD": (80, None, None), "PART_OFF": (85, 0.06, 2.5),
        "CHAMFER": (180, 0.08, 0.5)
    },
    "ALUMINIUM": {
        "FACE": (500, 0.25, 1.5), "ROUGH": (600, 0.35, 3.0),
        "FINISH": (800, 0.15, 0.5), "GROOVE": (300, 0.10, 2.0),
        "THREAD": (200, None, None), "PART_OFF": (250, 0.10, 2.5),
        "CHAMFER": (700, 0.12, 0.5)
    },
    "STAINLESS": {
        "FACE": (110, 0.15, 0.8), "ROUGH": (120, 0.20, 1.5),
        "FINISH": (150, 0.09, 0.3), "GROOVE": (70, 0.05, 1.5),
        "THREAD": (60, None, None), "PART_OFF": (65, 0.05, 2.0),
        "CHAMFER": (140, 0.07, 0.4)
    }
}

# Tool library: {operation class: (station, ISO designation, description)}
TOOL_LIBRARY: Dict[str, tuple] = {
    "FACE": ("T0101", "DCLNR 2525 M12 + CNMG 120408-PR", "95 deg right-hand facing/turning tool, 0.8 mm nose"),
    "ROUGH": ("T0101", "DCLNR 2525 M12 + CNMG 120408-PR", "95 deg right-hand roughing tool, 0.8 mm nose"),
    "FINISH": ("T0202", "SDJCR 2525 M11 + DCMT 11T304-PF", "93 deg right-hand finishing tool, 0.4 mm nose"),
    "CHAMFER": ("T0202", "SDJCR 2525 M11 + DCMT 11T304-PF", "finishing tool used at 45 deg for chamfering"),
    "GROOVE": ("T0303", "QD-LCH 2525 + QD-NH-0300", "3 mm parting/grooving blade"),
    "THREAD": ("T0404", "SER 2525 M16 + 16ER AG60", "60 deg external threading insert"),
    "PART_OFF": ("T0303", "QD-LCH 2525 + QD-NH-0300", "3 mm parting blade"),
}

# Operation precedence - lower number is machined first.
PRECEDENCE: Dict[str, int] = {
    "FACE": 10, "ROUGH": 20, "GROOVE": 30, "FINISH": 40,
    "CHAMFER": 50, "THREAD": 60, "PART_OFF": 70,
}

# Thread infeed schedule: number of passes as a function of pitch (ISO metric).
THREAD_PASSES = {0.5: 5, 0.75: 6, 1.0: 7, 1.25: 8, 1.5: 9, 1.75: 10, 2.0: 11, 2.5: 13, 3.0: 15}

# -----
# 2. INPUT DATA MODEL
# -----

@dataclass
class Step:
    """One cylindrical step of the finished component."""
    diameter: float # finished diameter [mm]
    length: float # axial length of this step [mm]

@dataclass
class PartSpec:
    """Complete description of the turned component to be planned."""
    name: str
    material: str # key into CUTTING_DATA
    stock_diameter: float # bar diameter [mm]
    stock_length: float # length of bar used per piece [mm]
    steps: List[Step] # ordered from the free (chuck-remote) end
    facing_allowance: float = 1.0 # material removed when facing [mm]
    chamfer_size: float = 0.0 # chamfer leg length [mm], 0 = none
    chamfer_angle: float = 45.0 # chamfer angle [deg]
    thread_pitch: float = 0.0 # 0 = no thread
    thread_length: float = 0.0 # threaded length [mm]
    thread_on_step: int = 0 # index into steps that carries the thread
    groove_width: float = 0.0 # thread relief groove, 0 = none
    groove_depth: float = 0.0
    max_spindle_rpm: float = 3500.0 # machine limit
    max_constant_surface_rpm: float = 3500.0
    rapid_time_per_op: float = 0.08 # approach/retract allowance per op [min]
```

```

@dataclass
class Operation:
    """One planned operation, fully costed."""
    seq: int
    name: str
    op_class: str
    tool_station: str
    tool_desc: str
    diameter: float          # diameter used for the speed calculation [mm]
    Vc: float                # cutting speed [m/min]
    N: float                 # spindle speed [rev/min]
    feed: float              # feed [mm/rev]
    ap: Optional[float]      # depth of cut [mm]
    passes: int
    length: float            # cut length per pass [mm]
    time_min: float          # machining time including approach [min]
    note: str = ""

# -----
# 3. PROCESS CALCULATIONS
# -----

def spindle_speed(Vc: float, diameter: float, limit: float) -> float:
    """N = 1000*Vc / (pi*D), clamped to the machine maximum."""
    if diameter <= 0:
        return limit
    return min(1000.0 * Vc / (math.pi * diameter), limit)

def cut_time(length: float, feed: float, N: float) -> float:
    """Machining time for one pass, t = L / (f * N) [min]."""
    if feed <= 0 or N <= 0:
        return 0.0
    return length / (feed * N)

def thread_pass_count(pitch: float) -> int:
    """Number of infeed passes for a given ISO metric pitch."""
    if pitch in THREAD_PASSES:
        return THREAD_PASSES[pitch]
    # linear interpolation / extrapolation for non-tabulated pitches
    return max(4, int(round(4.0 + 3.6 * pitch)))

# -----
# 4. THE PLANNER
# -----

def recognise_features(part: PartSpec) -> List[dict]:
    """Classify the input geometry into machinable features."""
    feats: List[dict] = []

    feats.append({"cls": "FACE", "name": "Face the bar end",
                  "D": part.stock_diameter, "L": part.stock_diameter / 2.0,
                  "stock": part.facing_allowance})

    # Each finished step that is smaller than the stock needs roughing + finishing.
    z_from_end = 0.0
    for i, st in enumerate(part.steps):
        radial_stock = (part.stock_diameter - st.diameter) / 2.0
        if radial_stock > 0.01:
            feats.append({"cls": "ROUGH", "name": f"Rough turn step {i+1} to Ø{st.diameter:.1f}",
                          "D": part.stock_diameter, "L": st.length + z_from_end,
                          "stock": radial_stock, "D_final": st.diameter})
            z_from_end += st.length

    z_from_end = 0.0
    for i, st in enumerate(part.steps):
        if (part.stock_diameter - st.diameter) / 2.0 > 0.01:
            feats.append({"cls": "FINISH", "name": f"Finish turn step {i+1} to Ø{st.diameter:.1f}",
                          "D": st.diameter, "L": st.length + z_from_end, "stock": 0.0})
            z_from_end += st.length

    if part.groove_width > 0:
        gd = part.steps[part.thread_on_step].diameter
        feats.append({"cls": "GROOVE", "name": "Thread relief groove",
                      "D": gd, "L": part.groove_depth, "stock": part.groove_depth,
                      "width": part.groove_width})

    if part.chamfer_size > 0:
        d0 = part.steps[0].diameter
        feats.append({"cls": "CHAMFER", "name": f"Chamfer {part.chamfer_size}x{part.chamfer_angle:.0f}°",
                      "D": d0, "L": part.chamfer_size / math.sin(math.radians(part.chamfer_angle)),
                      "stock": 0.0})

    if part.thread_pitch > 0:
        td = part.steps[part.thread_on_step].diameter
        feats.append({"cls": "THREAD",
                      "name": f"Thread M{td:.0f}x{part.thread_pitch} over {part.thread_length} mm",
                      "D": td, "L": part.thread_length, "stock": 0.0,
                      "pitch": part.thread_pitch})

    feats.append({"cls": "PART_OFF", "name": "Part off component",
                  "D": part.steps[-1].diameter if part.steps else part.stock_diameter,
                  "L": (part.steps[-1].diameter if part.steps else part.stock_diameter) / 2.0,
                  "stock": 0.0})

    return feats

def plan(part: PartSpec) -> List[Operation]:
    """Generate the complete, ordered and costed process plan."""
    if part.material not in CUTTING_DATA:
        raise ValueError(f"No cutting data for material '{part.material}'. "
                        f"Known: {sorted(CUTTING_DATA)}")

    feats = recognise_features(part)
    # stable sort by the precedence rules
    feats.sort(key=lambda f: PRECEDENCE[f["cls"]])

    data = CUTTING_DATA[part.material]
    ops: List[Operation] = []

    for i, f in enumerate(feats, start=1):
        cls = f["cls"]
        Vc, feed, ap = data[cls]
        station, iso, desc = TOOL_LIBRARY[cls]

```

```

note = ""

if cls == "THREAD":
    pitch = f["pitch"]
    passes = thread_pass_count(pitch)
    feed = pitch # threading feed always equals the pitch
    N = spindle_speed(Vc, f["D"], part.max_spindle_rpm)
    t = passes * cut_time(f["L"] + 4.0 * pitch, feed, N) # +run-in/run-out
    note = f"{passes} infeed passes, feed = {pitch} mm/rev"
    ap_used = None
elif cls == "ROUGH":
    passes = max(1, math.ceil(f["stock"] / ap))
    # Speed is computed on the mean diameter actually being cut.
    d_mean = (f["D"] + f.get("D final", f["D"])) / 2.0
    N = spindle_speed(Vc, d_mean, part.max_spindle_rpm)
    t = passes * cut_time(f["L"], feed, N)
    note = f"radial stock {f['stock']:.2f} mm / ap {ap} mm"
    ap_used = ap
elif cls == "FACE":
    passes = max(1, math.ceil(f["stock"] / ap))
    N = spindle_speed(Vc, f["D"], part.max_spindle_rpm)
    t = passes * cut_time(f["L"], feed, N)
    note = "G94 facing cycle, constant surface speed recommended"
    ap_used = ap
elif cls == "GROOVE":
    passes = max(1, math.ceil(f["width"] / 3.0)) # 3 mm wide blade
    N = spindle_speed(Vc, f["D"], part.max_spindle_rpm)
    t = passes * cut_time(f["L"], feed, N)
    note = f"groove {f['width']} mm wide x {f['stock']} mm deep"
    ap_used = f["stock"]
else:
    # FINISH, CHAMFER, PART_OFF
    passes = 1
    N = spindle_speed(Vc, f["D"], part.max_spindle_rpm)
    t = cut_time(f["L"], feed, N)
    ap_used = ap

ops.append(Operation(seq=i, name=f["name"], op_class=cls,
                    tool_station=station, tool_desc=f"{iso} - {desc}",
                    diameter=f["D"], Vc=Vc, N=N, feed=feed, ap=ap_used,
                    passes=passes, length=f["L"],
                    time_min=t + part.rapid_time_per_op, note=note))

return ops

# -----
# 5. REPORTING
# -----

def print_plan(part: PartSpec, ops: List[Operation]) -> float:
    print("=" * 100)
    print(f"GENERATIVE PROCESS PLAN - {part.name}")
    print("=" * 100)
    print(f"Work material      : {part.material}")
    print(f"Stock                  : Ø{part.stock_diameter} x {part.stock_length} mm bar")
    print(f"Finished steps        : " +
          ", ".join(f"Ø{s.diameter}x{s.length}" for s in part.steps))
    if part.thread_pitch:
        print(f"Thread                  : pitch {part.thread_pitch} mm over {part.thread_length} mm")
    if part.chamfer_size:
        print(f"Chamfer                 : {part.chamfer_size} mm x {part.chamfer_angle}°")
    print("-" * 100)
    print(f"{'#':>2} {'Operation':<34}{'Tool':<7}{'Vc':>5} {'N':>6} {'f':>6} {'ap':>5} {'x':>3} {'t,min':>7}")
    print("-" * 100)

    total = 0.0
    for op in ops:
        ap_s = f"{op.ap:.2f}" if op.ap is not None else " - "
        print(f"{op.seq:>2} {op.name[:33]:<34}{op.tool_station:<7}"
              f"{op.Vc:>5.0f} {op.N:>6.0f} {op.feed:>6.3f} {ap_s:>5} "
              f"{op.passes:>3} {op.time_min:>7.3f}")
        total += op.time_min

    print("-" * 100)
    print(f"{':':>2} {'TOTAL CYCLE TIME (machining + approach)':<34}{'':<7}{'':>5} {'':>6} {'':>6} {'':>5} {'':>3}"
          f"{total:>7.3f}")
    print(f"{':':>2} {'equivalent':<34} = {total*60:.1f} seconds per component")
    print("-" * 100)
    print("Tooling list")
    for st in sorted({o.tool_station: o.tool_desc for o in ops}.items()):
        print(f"    {st[0]} : {st[1]}")
    print("Operation notes")
    for op in ops:
        if op.note:
            print(f"    [{op.seq}] {op.note}")
    print()
    return total

# -----
# 6. TEST CASES
# -----

def test_case_1() -> PartSpec:
    """Stepped shaft with an M30x1.5 thread - mild steel."""
    return PartSpec(
        name="TEST CASE 1 - Stepped transmission shaft (MS)",
        material="MILD STEEL", stock_diameter=50.0, stock_length=100.0,
        steps=[Step(30.0, 25.0), Step(40.0, 30.0), Step(50.0, 40.0)],
        facing_allowance=1.0, chamfer_size=2.0, chamfer_angle=45.0,
        thread_pitch=1.5, thread_length=20.0, thread_on_step=0,
        groove_width=4.0, groove_depth=2.0, max_spindle_rpm=3500.0)

def test_case_2() -> PartSpec:
    """Small aluminium pin, no thread - checks the no-thread/no-groove branch."""
    return PartSpec(
        name="TEST CASE 2 - Aluminium locating pin (no thread)",
        material="ALUMINIUM", stock_diameter=25.0, stock_length=60.0,
        steps=[Step(12.0, 20.0), Step(20.0, 25.0)],
        facing_allowance=0.8, chamfer_size=1.0, chamfer_angle=45.0,
        thread_pitch=0.0, thread_length=0.0,
        groove_width=0.0, groove_depth=0.0, max_spindle_rpm=6000.0)

def test_case_3() -> PartSpec:
    """Heavy stainless flange stub - large radial stock, coarse 2.5 mm thread."""
    return PartSpec(
        name="TEST CASE 3 - Stainless valve stem (heavy stock)",
        material="STAINLESS", stock_diameter=90.0, stock_length=150.0,

```

```

steps=[Step(48.0, 40.0), Step(70.0, 45.0), Step(90.0, 50.0)],
facing_allowance=1.5, chamfer_size=2.5, chamfer_angle=45.0,
thread_pitch=2.5, thread_length=35.0, thread_on_step=0,
groove_width=6.0, groove_depth=3.0, max_spindle_rpm=2500.0)

def main() -> None:
    print("\n" + "#" * 100)
    print("# TASK 3.1 - GENERATIVE CAPP FEATURE RECOGNIZER : TEST VERIFICATION RUN")
    print("#" * 100 + "\n")

    results = []
    for builder in (test_case_1, test_case_2, test_case_3):
        part = builder()
        ops = plan(part)
        total = print_plan(part, ops)
        results.append((part.name, len(ops), total))

    # --- automatic sanity checks (assertions act as the verification) ---
    assert ops[0].op_class == "FACE", "facing must be the first operation"
    assert ops[-1].op_class == "PART OFF", "part-off must be the last operation"
    order = [PRECEDENCE[o.op_class] for o in ops]
    assert order == sorted(order), "operations are not in precedence order"
    assert all(o.N > 0 for o in ops), "a spindle speed came out non-positive"
    assert all(o.N <= part.max_spindle_rpm + 1e-6 for o in ops), "spindle limit exceeded"
    assert total > 0, "total time must be positive"
    if part.thread_pitch:
        th = [o for o in ops if o.op_class == "THREAD"][0]
        assert abs(th.feed - part.thread_pitch) < 1e-9, "threading feed must equal the pitch"
    print("    >>> SANITY CHECKS PASSED for this case\n")

    print("#" * 100)
    print("SUMMARY OF THE THREE TEST CASES")
    print("#" * 100)
    print(f"{'Case':<50}{'Ops':>6}{'Cycle time, min':>18}")
    for name, n, t in results:
        print(f"{'name':<50}{'n':>6}{'t':>18.3f}")
    print("#" * 100)
    print("ALL TEST CASES COMPLETED SUCCESSFULLY")

if __name__ == "__main__":
    main()

```

```
#!/usr/bin/env python3
"""
Task 3.2 - CNC Tool Offset & Cutter Radius Compensation Calculator
=====
Calculates the corrected tool-path coordinates for a given part profile when a
tool with a finite nose radius / cutter radius is used.

Two cases are handled with the same underlying geometry engine:

    TURNING (X, Z plane) - tool nose radius compensation (G41/G42 on a lathe).
                          The programmed point on a lathe is the *imaginary
                          tool nose*; the actual cutting point wanders around
                          the nose radius, which produces undercut on tapers
                          and arcs unless compensation is applied.

    MILLING (X, Y plane) - cutter radius compensation (G41 = cutter LEFT of the
                          programmed path, G42 = cutter RIGHT).

Geometry engine
-----
The compensated path is the *offset* of the part profile by the tool radius,
taken on the side the tool body lies. For a polyline this is computed by

    1. offsetting every straight segment perpendicular to itself by r, and
    2. intersecting each pair of neighbouring offset lines (a mitre joint).

Parallel neighbours (collinear segments) fall back to the plain perpendicular
offset, which is the correct limit of the mitre.

Run: python3 task3_2_tool_offset_compensation.py
"""

from __future__ import annotations

import math
from typing import List, Tuple, Sequence

Pt = Tuple[float, float]
EPS = 1e-9

# -----
# 1. BASIC 2-D GEOMETRY
# -----

def _sub(a: Pt, b: Pt) -> Pt:
    return (a[0] - b[0], a[1] - b[1])

def _add(a: Pt, b: Pt) -> Pt:
    return (a[0] + b[0], a[1] + b[1])

def _scale(a: Pt, k: float) -> Pt:
    return (a[0] * k, a[1] * k)

def _norm(a: Pt) -> float:
    return math.hypot(a[0], a[1])

def _unit(a: Pt) -> Pt:
    n = _norm(a)
    if n < EPS:
        raise ValueError("zero-length segment in the profile")
    return (a[0] / n, a[1] / n)

def left_normal(d: Pt) -> Pt:
    """Unit normal 90 deg to the LEFT of direction d (d must be a unit vector)."""
    return (-d[1], d[0])

def right_normal(d: Pt) -> Pt:
    """Unit normal 90 deg to the RIGHT of direction d."""
    return (d[1], -d[0])

def line_intersection(p1: Pt, d1: Pt, p2: Pt, d2: Pt) -> Pt | None:
    """Intersection of line (p1 + t*d1) with line (p2 + s*d2); None if parallel."""
    den = d1[0] * d2[1] - d1[1] * d2[0]
    if abs(den) < 1e-12:
        return None # parallel or collinear
    dx, dy = p2[0] - p1[0], p2[1] - p1[1]
    t = (dx * d2[1] - dy * d2[0]) / den
    return (p1[0] + t * d1[0], p1[1] + t * d1[1])

def point_to_segment_distance(p: Pt, a: Pt, b: Pt) -> float:
    """Shortest distance from point p to the segment a-b."""
    ab = _sub(b, a)
    L2 = ab[0]**2 + ab[1]**2
    if L2 < EPS:
        return _norm(_sub(p, a))
    t = max(0.0, min(1.0, ((p[0] - a[0]) * ab[0] + (p[1] - a[1]) * ab[1]) / L2))
    proj = (a[0] + t * ab[0], a[1] + t * ab[1])
    return _norm(_sub(p, proj))

def distance_to_polyline(p: Pt, poly: Sequence[Pt]) -> float:
    return min(point_to_segment_distance(p, poly[i], poly[i + 1])
               for i in range(len(poly) - 1))

# -----
# 2. THE OFFSET (COMPENSATION) ENGINE
# -----

def offset_polyline(pts: Sequence[Pt], r: float, side: str,
                    closed: bool = False) -> List[Pt]:
    """
    Offset an open or closed polyline by a distance r.
    side : 'left' -> offset to the left of the direction of travel (G41)
    """
```

```

'right' -> offset to the right of the direction of travel (G42)

Returns the offset polyline with mitred corners.
"""
if side not in ("left", "right"):
    raise ValueError("side must be 'left' or 'right'")
if len(pts) < 2:
    raise ValueError("a profile needs at least two points")

# Remove repeated points - a duplicated vertex would give a zero-length
# segment with no defined direction. This also tolerates a caller that
# already closed the contour by repeating the first point at the end.
clean: List[Pt] = [tuple(pts[0])]
for p in pts[1:]:
    if _norm(_sub(p, clean[-1])) > 1e-9:
        clean.append(tuple(p))
pts = clean
if closed and _norm(_sub(pts[0], pts[-1])) > 1e-9:
    pts.append(pts[0])
elif not closed and len(pts) < 2:
    raise ValueError("a profile needs at least two distinct points")

nrm = left_normal if side == "left" else right_normal

# one offset line per segment: (point on the line, direction)
lines = []
for i in range(len(pts) - 1):
    d = unit(_sub(pts[i + 1], pts[i]))
    n = nrm(d)
    lines.append((_add(pts[i], _scale(n, r)), d, n))

out: List[Pt] = []

if not closed:
    out.append(lines[0][0]) # start of the first offset line

rng = range(len(lines)) if closed else range(1, len(lines))
for i in rng:
    prev = lines[i - 1]
    cur = lines[i]
    ip = line_intersection(prev[0], prev[1], cur[0], cur[1])
    if ip is None: # collinear segments
        ip = _add(pts[i], _scale(cur[2], r))
    out.append(ip)

if not closed:
    last_p, last_d, last_n = lines[-1]
    out.append(_add(pts[-1], _scale(last_n, r)))
else:
    out.append(out[0]) # close the contour

return out

# -----
# 3. TURNING - TOOL NOSE RADIUS COMPENSATION
# -----

# Imaginary tool-nose direction for the usual Fanuc tip numbers, expressed as a
# unit vector (Z, X) pointing from the CENTRE of the nose radius towards the
# imaginary tip. NOTE: the sign convention depends on whether the machine has a
# front or a rear tool post - always confirm against the control manual.
TIP_VECTORS = {
    0: (0.0, 0.0), # nose centre programmed directly
    1: (+0.70710678, +0.70710678), # +Z, +X
    2: (-0.70710678, +0.70710678), # -Z, +X
    3: (-0.70710678, -0.70710678), # -Z, -X
    4: (+0.70710678, -0.70710678), # +Z, -X
    5: (0.0, +1.0), # +X
    6: (-1.0, 0.0), # -Z
    7: (0.0, -1.0), # -X
    8: (+1.0, 0.0), # +Z
}

def turning_compensation(profile: Sequence[Pt], nose_radius: float,
                        tip_vector: Pt, material_side: str = "left"):
    """
    Compute the compensated path for a turned profile.

    profile : list of (Z, X radius) points describing the finished surface
    nose_radius : R of the insert [mm]
    tip_vector : unit vector (Z, X) from the nose centre to the imaginary tip
    material_side : which side of the direction of travel the MATERIAL lies on;
                   the tool is therefore offset to the opposite side

    Returns (centre_path, tip_path, max uncompensated error)
    centre_path - path of the CENTRE of the nose radius (true offset)
    tip_path - where the imaginary tip must be commanded with G40
    error - the largest deviation that would occur if the profile were
            programmed directly on the tip with no compensation
    """
    tool_side = "right" if material_side == "left" else "left"
    centre_path = offset_polyline(profile, nose_radius, tool_side)

    # The imaginary tip sits at (centre + R * tip_vector).
    tip_path = [_add(c, _scale(tip_vector, nose_radius)) for c in centre_path]

    # If the operator ignored compensation he would drive the TIP along the
    # profile; the nose centre would then be at (profile - R*tip_vector) and the
    # surface actually produced is that centre path offset back by R. The error
    # is the distance between the resulting surface and the wanted profile.
    err = 0.0
    for p in profile:
        wrong_centre = _sub(p, _scale(tip_vector, nose_radius))
        d = distance_to_polyline(wrong_centre, profile)
        err = max(err, abs(d - nose_radius))
    return centre_path, tip_path, err

# -----
# 4. MILLING - CUTTER RADIUS COMPENSATION
# -----

def milling_compensation(contour: Sequence[Pt], cutter_diameter: float,
                        gcode: str, closed: bool = True) -> List[Pt]:
    """
    Cutter radius compensation for a milled contour.
    """

```

```

gcode : 'G41' -> cutter kept to the LEFT of the direction of travel
       'G42' -> cutter kept to the RIGHT of the direction of travel
"""
gcode = gcode.upper()
if gcode not in ("G41", "G42"):
    raise ValueError("gcode must be 'G41' or 'G42'")
side = "left" if gcode == "G41" else "right"
return offset_polyline(contour, cutter_diameter / 2.0, side, closed=closed)

def arc_to_polyline(centre: Pt, radius: float, start_deg: float,
                    end_deg: float, segments: int = 72) -> List[Pt]:
    """Chord the arc so that circular contours can use the same engine."""
    pts = []
    for i in range(segments + 1):
        a = math.radians(start_deg + (end_deg - start_deg) * i / segments)
        pts.append((centre[0] + radius * math.cos(a),
                    centre[1] + radius * math.sin(a)))
    return pts

# -----
# 5. REPORTING
# -----

def show(title: str, header: Tuple[str, str], original: Sequence[Pt],
         compensated: Sequence[Pt], limit: int = 12) -> None:
    print("-" * 78)
    print(title)
    print("-" * 78)
    a, b = header
    print(f"{'#':>3} {'PROGRAMMED ' + a:>14} {'PROGRAMMED ' + b:>14}")
    print(f"{'#':>3} {'COMPENSATED ' + a:>16} {'COMPENSATED ' + b:>16}")
    n = max(len(original), len(compensated))
    for i in range(min(n, limit)):
        o = original[i] if i < len(original) else ("", "")
        c = compensated[i] if i < len(compensated) else ("", "")
        os0 = f"{o[0]:14.4f}" if o[0] != "" else " " * 14
        os1 = f"{o[1]:14.4f}" if o[1] != "" else " " * 14
        cs0 = f"{c[0]:16.4f}" if c[0] != "" else " " * 16
        cs1 = f"{c[1]:16.4f}" if c[1] != "" else " " * 16
        print(f"{'#':>3} {os0}{os1} {'':>4}{cs0}{cs1}")
    if n > limit:
        print(f"          ... {n - limit} further points not shown")
    print()

# -----
# 6. TEST CASES
# -----

def test_case_1() -> None:
    """Turning: cylinder + 30 deg taper + cylinder, 0.8 mm nose radius."""
    print("-" * 78)
    print("TEST CASE 1 - TURNING, tool nose radius compensation")
    print("-" * 78)
    # profile in (Z, X radius); cutting from the free end towards the chuck
    profile = [(0.0, 15.0), (-20.0, 15.0), (-40.0, 25.0), (-70.0, 25.0)]
    R = 0.8
    tip = TIP_VECTORS[3]  # tip points -Z, -X (typical OD turning)

    centre, tip_path, err = turning_compensation(profile, R, tip, material_side="left")

    print(f"  Insert nose radius R      : {R} mm")
    print(f"  Imaginary tip vector (Z,X): ({tip[0]:+.4f}, {tip[1]:+.4f}) [Fanuc tip no. 3]")
    print(f"  Taper angle in the profile: {math.degrees(math.atan2(25-15, 20)):.2f} deg")
    show("Nose-centre path (what the control actually drives with G41/G42)",
        ("Z", "X"), profile, centre)
    show("Imaginary-tip path (what must be programmed if G40, i.e. no comp)",
        ("Z", "X"), profile, tip_path)
    print(f">>> Maximum profile error if compensation were OMITTED : {err:.4f} mm")
    print(f"(this is the classic undercut on the tapered face)\n")

    # verification
    # The nose centre must never come CLOSER than R to the finished surface
    # (that would be a gouge); at convex mitre corners it is legitimately farther.
    for p in centre:
        d = distance_to_polyline(p, profile)
        assert d >= R - 1e-6, f"gouge: nose centre only {d:.4f} mm from the profile"
    assert err > 0.01, "a taper must produce a measurable uncompensated error"
    print(">>> SANITY CHECKS PASSED\n")

def test_case_2() -> None:
    """Milling: rectangular pocket, inside contour, 10 mm cutter."""
    print("-" * 78)
    print("TEST CASE 2 - MILLING, rectangular pocket (G41, inside contour)")
    print("-" * 78)
    W, Hh = 60.0, 40.0
    contour = [(0.0, 0.0), (W, 0.0), (W, Hh), (0.0, Hh)]  # CCW, closed
    Dc = 10.0

    comp = milling_compensation(contour, Dc, "G41", closed=True)

    print(f"  Pocket size      : {W} x {Hh} mm")
    print(f"  Cutter diameter  : {Dc} mm (radius {Dc/2} mm)")
    show("Cutter-centre path", ("X", "Y"), contour, comp)

    xs = [p[0] for p in comp[:-1]]
    ys = [p[1] for p in comp[:-1]]
    got_w, got_h = max(xs) - min(xs), max(ys) - min(ys)
    exp_w, exp_h = W - Dc, Hh - Dc
    print(f"  Cutter-centre rectangle : {got_w:.4f} x {got_h:.4f} mm")
    print(f"  Expected (W-D) x (H-D) : {exp_w:.4f} x {exp_h:.4f} mm")
    assert abs(got_w - exp_w) < 1e-6 and abs(got_h - exp_h) < 1e-6, \
        "inside offset of a rectangle must shrink it by exactly one diameter"
    assert abs(min(xs) - Dc / 2) < 1e-6, "offset must start one radius in from the wall"
    print(">>> SANITY CHECKS PASSED - offset is exactly one radius from every wall\n")

def test_case_3() -> None:
    """Milling: outside profile with G42, and a circular pocket."""
    print("-" * 78)
    print("TEST CASE 3 - MILLING, outside contour (G42) and circular pocket")
    print("-" * 78)
    contour = [(0.0, 0.0), (80.0, 0.0), (80.0, 50.0), (40.0, 70.0), (0.0, 50.0)]
    Dc = 12.0
    comp = milling_compensation(contour, Dc, "G42", closed=True)

```

```

print(f" Cutter diameter : Ø{Dc} mm, outside contour, G42")
show("Cutter-centre path around the outside of the part", ("X", "Y"), contour, comp)

# every centre point must sit exactly one radius outside the part
for p in comp[:-1]:
    d = distance_to_polyline(p, list(contour) + [contour[0]])
    assert d > Dc / 2 - 1e-6, f"cutter centre too close to the part: {d:.4f}"
print(" >>> every cutter-centre point is at least one radius clear of the profile")

# circular pocket
Dpocket, Dcut = 40.0, 10.0
circ = arc_to_polyline((0.0, 0.0), Dpocket / 2.0, 0.0, 360.0, segments=36)
cc = milling_compensation(circ, Dcut, "G41", closed=True)
radii = [math.hypot(p[0], p[1]) for p in cc[:-1]]
print(f"\n Circular pocket Ø{Dpocket} mm with a Ø{Dcut} mm cutter")
print(f" Cutter-centre radius : min {min(radii):.4f} / max {max(radii):.4f} mm")
print(f" Expected : {(Dpocket - Dcut)/2:.4f} mm")
assert abs(sum(radii)/len(radii) - (Dpocket - Dcut) / 2) < 0.05, \
    "circular pocket offset radius is wrong"
print(" >>> SANITY CHECKS PASSED\n")

def main() -> None:
    print("\n" + "#" * 78)
    print("# TASK 3.2 - TOOL OFFSET / CUTTER RADIUS COMPENSATION : VERIFICATION RUN")
    print("#" * 78 + "\n")
    test_case_1()
    test_case_2()
    test_case_3()
    print("=" * 78)
    print("ALL THREE TEST CASES COMPLETED SUCCESSFULLY")
    print("=" * 78)

if __name__ == "__main__":
    main()

```

```
#!/usr/bin/env python3
"""
Task 3.3 - AS/RS Single- and Dual-Command Cycle Time Algorithm
=====
Engineering analysis of a single-rack (one aisle, one S/R machine) Automated
Storage & Retrieval System. The program computes single-command (SC) and
dual-command (DC) travel cycle times and the resulting throughput for
    * RANDOMIZED storage - analytical Bozer & White model, and
    * DEDICATED storage - explicit travel to named rack locations.

Theory
-----
The S/R machine drives its horizontal (y) and vertical (z) axes SIMULTANEOUSLY,
so the travel time between two points is the CHEBYSHEV time

    t = max( dy / Vy , dz / Vz )

Define the time to reach the far corner of the rack along each axis

    th = L / Vy      tv = H / Vz
    T = max(th, tv)  scaling factor
    b = min(th, tv) / T      rack shape factor, 0 < b <= 1

For randomized storage with the input/output point at one lower corner, Bozer
and White give the expected travel times

    E(SC) = T * ( 1 + b^2 / 3 )
    E(DC) = T * ( 4/3 + b^2 / 2 - b^3 / 30 )

E(SC) is a complete round trip (I/O point -> location -> I/O point).
E(DC) covers I/O -> storage location -> retrieval location -> I/O.

Adding the pick-and-deposit (P/D) time tpd:

    SC cycle = E(SC) + 2 * tpd      -> 1 transaction
    DC cycle = E(DC) + 4 * tpd      -> 2 transactions

The analytical SC result is re-derived inside this program by Monte-Carlo
simulation, which is used as an independent verification of the formula.

Run: python3 task3_3_asrs_cycle_time.py
"""

from __future__ import annotations
import math
import random
from dataclasses import dataclass
from typing import Tuple, List

# -----
# 1. SYSTEM DEFINITION
# -----

@dataclass
class ASRS:
    """Geometry and kinematics of one AS/RS aisle."""
    name: str
    rack_length: float      # L [m]
    rack_height: float      # H [m]
    v_horizontal: float      # Vy [m/s]
    v_vertical: float        # Vz [m/s]
    tpd: float              # pick-and-deposit time per operation [s]
    availability: float = 1.0 # fraction of the hour the machine is available

    # ---- derived quantities -----
    @property
    def th(self) -> float:
        """Time to traverse the full rack LENGTH [s]."""
        return self.rack_length / self.v_horizontal

    @property
    def tv(self) -> float:
        """Time to traverse the full rack HEIGHT [s]."""
        return self.rack_height / self.v_vertical

    @property
    def T(self) -> float:
        """Scaling factor T = max(th, tv) [s]."""
        return max(self.th, self.tv)

    @property
    def b(self) -> float:
        """Shape factor b = min(th,tv)/T, 0 < b <= 1 (b = 1 is a square-in-time rack)."""
        return min(self.th, self.tv) / self.T

    def travel_time(self, p1: Tuple[float, float], p2: Tuple[float, float]) -> float:
        """Chebyshev travel time between two (y, z) rack positions [s]."""
        return max(abs(p2[0] - p1[0]) / self.v_horizontal,
                    abs(p2[1] - p1[1]) / self.v_vertical)

# -----
# 2. RANDOMIZED STORAGE - ANALYTICAL MODEL
# -----

def sc_travel_random(s: ASRS) -> float:
    """Expected SINGLE-COMMAND travel time, randomized storage [s]."""
    return s.T * (1.0 + s.b ** 2 / 3.0)

def dc_travel_random(s: ASRS) -> float:
    """Expected DUAL-COMMAND travel time, randomized storage [s]."""
    return s.T * (4.0 / 3.0 + s.b ** 2 / 2.0 - s.b ** 3 / 30.0)

def sc_cycle_random(s: ASRS) -> float:
    """Complete SC cycle time including P/D handling [s]."""
    return sc_travel_random(s) + 2.0 * s.tpd

def dc_cycle_random(s: ASRS) -> float:
    """Complete DC cycle time including P/D handling [s]."""
```

```

return dc_travel_random(s) + 4.0 * s.tpd

def throughput(s: ASRS, cycle_s: float, transactions_per_cycle: int) -> float:
    """Transactions per hour for a given cycle time."""
    if cycle_s <= 0:
        return 0.0
    return 3600.0 * s.availability * transactions_per_cycle / cycle_s

# -----
# 3. RANDOMIZED STORAGE - MONTE-CARLO VERIFICATION
# -----

def sc_travel_monte_carlo(s: ASRS, n: int = 400_000, seed: int = 12345) -> float:
    """
    Independent estimate of E(SC) by simulation.
    A storage location is drawn uniformly over the rack face and the machine
    makes a round trip from the I/O point at the lower corner (0, 0).
    """
    rng = random.Random(seed)
    total = 0.0
    for _ in range(n):
        y = rng.uniform(0.0, s.rack_length)
        z = rng.uniform(0.0, s.rack_height)
        total += 2.0 * s.travel_time((0.0, 0.0), (y, z))
    return total / n

def dc_travel_monte_carlo(s: ASRS, n: int = 400_000, seed: int = 999) -> float:
    """
    Independent estimate of E(DC): I/O -> store at P1 -> travel to P2 -> I/O.
    Both locations are drawn uniformly and independently.
    """
    rng = random.Random(seed)
    total = 0.0
    io = (0.0, 0.0)
    for _ in range(n):
        p1 = (rng.uniform(0, s.rack_length), rng.uniform(0, s.rack_height))
        p2 = (rng.uniform(0, s.rack_length), rng.uniform(0, s.rack_height))
        total += (s.travel_time(io, p1) + s.travel_time(p1, p2) + s.travel_time(p2, io))
    return total / n

# -----
# 4. DEDICATED STORAGE - EXPLICIT LOCATIONS
# -----

@dataclass
class Location:
    """A named, dedicated rack location."""
    label: str
    y: float          # distance along the rack [m]
    z: float          # height [m]

    def as_tuple(self) -> Tuple[float, float]:
        return (self.y, self.z)

def sc_cycle_dedicated(s: ASRS, loc: Location) -> float:
    """SC cycle to one dedicated location (round trip + 2 P/D) [s]."""
    return 2.0 * s.travel_time((0.0, 0.0), loc.as_tuple()) + 2.0 * s.tpd

def dc_cycle_dedicated(s: ASRS, store: Location, retrieve: Location) -> float:
    """DC cycle: I/O -> store -> retrieve -> I/O, plus 4 P/D [s]."""
    io = (0.0, 0.0)
    t = (s.travel_time(io, store.as_tuple())
        + s.travel_time(store.as_tuple(), retrieve.as_tuple())
        + s.travel_time(retrieve.as_tuple(), io))
    return t + 4.0 * s.tpd

# -----
# 5. REPORTING
# -----

def analyse(s: ASRS, locations: List[Location] | None = None,
            monte_carlo: bool = True) -> dict:
    print("=" * 84)
    print(f"AS/RS CYCLE-TIME ANALYSIS - {s.name}")
    print("=" * 84)
    print(f" Rack length L = {s.rack_length:8.2f} m      Horizontal speed Vy = {s.v_horizontal:5.2f} m/s")
    print(f" Rack height H = {s.rack_height:8.2f} m      Vertical speed Vz = {s.v_vertical:5.2f} m/s")
    print(f" P/D time tpd = {s.tpd:8.2f} s      Availability = {s.availability*100:5.1f} %")
    print("=" * 84)
    print(f" th = L/Vy = {s.th:6.2f} s      tv = H/Vz = {s.tv:6.2f} s")
    print(f" T = max(th,tv) = {s.T:6.2f} s      b = min/T = {s.b:6.4f}")
    print("=" * 84)

    e_sc, e_dc = sc_travel_random(s), dc_travel_random(s)
    c_sc, c_dc = sc_cycle_random(s), dc_cycle_random(s)
    r_sc = throughput(s, c_sc, 1)
    r_dc = throughput(s, c_dc, 2)

    print(" RANDOMIZED STORAGE (Bozer & White)")
    print(f" E(SC) travel = T(1 + b^2/3) = {e_sc:8.2f} s")
    print(f" E(DC) travel = T(4/3 + b^2/2 - b^3/30) = {e_dc:8.2f} s")
    print(f" SC cycle (+2 tpd) = {c_sc:8.2f} s -> {r_sc:7.1f} transactions/h")
    print(f" DC cycle (+4 tpd) = {c_dc:8.2f} s -> {r_dc:7.1f} transactions/h")
    print(f" Throughput gain of dual over single command = {(r_dc/r_sc - 1)*100:6.1f} %")

    out = {"T": s.T, "b": s.b, "E_SC": e_sc, "E_DC": e_dc,
           "SC_cycle": c_sc, "DC_cycle": c_dc, "R_SC": r_sc, "R_DC": r_dc}

    if monte_carlo:
        mc_sc = sc_travel_monte_carlo(s)
        mc_dc = dc_travel_monte_carlo(s)
        esc = abs(mc_sc - e_sc) / e_sc * 100
        edc = abs(mc_dc - e_dc) / e_dc * 100
        print("=" * 84)
        print(" MONTE-CARLO VERIFICATION OF THE ANALYTICAL FORMULAE (400 000 trials)")
        print(f" E(SC): analytical {e_sc:7.3f} s simulated {mc_sc:7.3f} s deviation {esc:5.2f} %")
        print(f" E(DC): analytical {e_dc:7.3f} s simulated {mc_dc:7.3f} s deviation {edc:5.2f} %")
        assert esc < 1.0, "SC model does not agree with the simulation"
        assert edc < 1.0, "DC model does not agree with the simulation"
        print(" >>> both models agree with the simulation to better than 1 %")
        out["MC_SC"], out["MC_DC"] = mc_sc, mc_dc

```

```

if locations:
    print("-" * 84)
    print(" DEDICATED STORAGE - named locations")
    print(f" {'Location':<14}{ 'y, m':>8}{ 'z, m':>8}{ 'SC cycle, s':>14}{ 'SC rate, /h':>14}")
    for loc in locations:
        c = sc_cycle_dedicated(s, loc)
        print(f" {'loc.label':<14}{loc.y:>8.2f}{loc.z:>8.2f}{c:>14.2f}{3600.0*s.availability/c:>14.1f}")
    if len(locations) >= 2:
        d = dc_cycle_dedicated(s, locations[0], locations[1])
        print(f" DC cycle {'locations[0].label'} (store) -> {'locations[1].label'} (retrieve)"
              f" = {d:.2f} s -> {2*3600.0*s.availability/d:.1f} transactions/h")
        out["DC dedicated"] = d
    print("-" * 84 + "\n")
    return out

# -----
# 6. TEST CASES
# -----

def main() -> None:
    print("\n" + "-" * 84)
    print("# TASK 3.3 - AS/RS CYCLE TIME ALGORITHM : TEST VERIFICATION RUN")
    print("#" * 84 + "\n")

    # ---- TEST CASE 1 : tall narrow rack, vertical axis dominates -----
    c1 = ASRS(name="TEST CASE 1 - Tall unit-load rack (vertical dominant)",
              rack_length=60.0, rack_height=18.0,
              v_horizontal=2.0, v_vertical=0.5, tpd=15.0, availability=0.95)
    locs1 = [Location("A-01-01", 3.0, 1.2), Location("A-30-08", 45.0, 14.4),
             Location("A-20-05", 30.0, 9.0)]
    r1 = analyse(c1, locs1)
    assert 0 < c1.b <= 1.0, "shape factor out of range"
    assert r1["DC cycle"] > r1["SC cycle"], "a DC cycle must take longer than an SC cycle"
    assert r1["R_DC"] > r1["R_SC"], "DC must give a higher throughput"

    # ---- TEST CASE 2 : long low rack, horizontal axis dominates -----
    c2 = ASRS(name="TEST CASE 2 - Long mini-load rack (horizontal dominant)",
              rack_length=90.0, rack_height=6.0,
              v_horizontal=1.2, v_vertical=0.8, tpd=8.0, availability=0.90)
    locs2 = [Location("B-05-02", 12.0, 2.0), Location("B-40-06", 72.0, 5.0)]
    r2 = analyse(c2, locs2)
    assert abs(c2.th - 75.0) < 1e-9, "th calculation is wrong"
    assert abs(c2.tv - 7.5) < 1e-9, "tv calculation is wrong"
    assert abs(c2.T - 75.0) < 1e-9 and abs(c2.b - 0.1) < 1e-9, "T/b calculation is wrong"

    # ---- TEST CASE 3 : square-in-time rack, b = 1 -----
    # L/Vy = H/Vz exactly, so b = 1 and the formulae take their maximum values
    c3 = ASRS(name="TEST CASE 3 - Square-in-time rack (b = 1, limiting case)",
              rack_length=40.0, rack_height=10.0,
              v_horizontal=2.0, v_vertical=0.5, tpd=10.0, availability=1.0)
    r3 = analyse(c3, [Location("C-01-01", 0.0, 0.0), Location("C-MAX", 40.0, 10.0)])
    assert abs(c3.b - 1.0) < 1e-9, "this rack should be square in time (b = 1)"
    # with b = 1 : E(SC) = T*(4/3 + 1/2 - 1/30) and E(DC) = T*(4/3 + 1/2 - 1/30)
    assert abs(r3["E_SC"] - c3.T * 4.0 / 3.0) < 1e-9, "b=1 SC limit is wrong"
    assert abs(r3["E_DC"] - c3.T * (4.0/3.0 + 0.5 - 1.0/30.0)) < 1e-9, "b=1 DC limit is wrong"
    print(" >>> b = 1 limiting values confirmed analytically\n")

    print("-" * 84)
    print("SUMMARY")
    print("-" * 84)
    print(f"{'Case':<52}{ 'b':>7}{ 'SC,s':>9}{ 'DC,s':>9}{ 'DC rate/h':>12}")
    for s, r in ((c1, r1), (c2, r2), (c3, r3)):
        print(f"{'s.name':<52}{r['b']:>7.4f}{r['SC cycle']:>9.2f}{r['DC cycle']:>9.2f}{r['R_DC']:>12.1f}")
    print("-" * 84)
    print("ALL TEST CASES COMPLETED SUCCESSFULLY")

if __name__ == "__main__":
    main()

```

```
#!/usr/bin/env python3
"""
Task 3.4 - Material Handling Fleet Sizing Model
=====
Computes the number of Automated Guided Vehicles required to satisfy a given
factory delivery demand.

Model (standard material-handling analysis, after Groover)
-----
For one delivery cycle a vehicle must

    load -> travel loaded -> unload -> travel empty back

so the DELIVERY CYCLE TIME is

    Tc = TL + Ld/Vc + TU + Le/Vc          [min per delivery]

    TL = loading time           [min]
    TU = unloading time         [min]
    Ld = loaded travel distance [m]
    Le = empty travel distance  [m]
    Vc = vehicle speed          [m/min]

The WORKLOAD imposed on the fleet by a demand of Rf deliveries per hour is

    WL = Rf * Tc          [min of vehicle work per hour]

A vehicle cannot work the full 60 minutes of an hour: it loses time to battery
charging and breakdowns (availability A) and to blocking, queueing and
congestion (traffic factor Ft <= 1). The AVAILABLE TIME per vehicle is

    AT = 60 * A * Ft      [min per hour per vehicle]

and the required fleet size is

    n = WL / AT           rounded UP to the next whole vehicle

The traffic factor may either be supplied directly or estimated from the number
of vehicles sharing the guide path, which makes the calculation implicit - the
program then solves it by iteration.

Run: python3 task3_4_agv_fleet_sizing.py
"""

from __future__ import annotations
import math
from dataclasses import dataclass
from typing import Optional, List

# -----
# 1. INPUT MODEL
# -----

@dataclass
class AGVSystem:
    """All inputs needed to size an AGV fleet."""
    name: str
    load_time: float          # TL [min]
    unload_time: float        # TU [min]
    loaded_distance: float    # Ld [m]
    empty_distance: float     # Le [m]
    velocity: float           # Vc [m/min]
    deliveries_per_hour: float # Rf [deliveries/h]
    availability: float = 0.95 # A, fraction
    traffic_factor: Optional[float] = 0.90 # Ft; None -> estimate it

    def __post_init__(self) -> None:
        if self.velocity <= 0:
            raise ValueError("vehicle velocity must be positive")
        if not 0 < self.availability <= 1:
            raise ValueError("availability must lie in (0, 1]")
        if self.traffic_factor is not None and not 0 < self.traffic_factor <= 1:
            raise ValueError("traffic factor must lie in (0, 1]")

# -----
# 2. CORE CALCULATIONS
# -----

def delivery_cycle_time(s: AGVSystem) -> dict:
    """Break the delivery cycle into its four components [min]."""
    t_loaded = s.loaded_distance / s.velocity
    t_empty = s.empty_distance / s.velocity
    Tc = s.load_time + t_loaded + s.unload_time + t_empty
    return {"load": s.load_time, "travel_loaded": t_loaded,
            "unload": s.unload_time, "travel_empty": t_empty, "Tc": Tc}

def workload(s: AGVSystem, Tc: float) -> float:
    """Total vehicle-minutes of work demanded per hour."""
    return s.deliveries_per_hour * Tc

def available_time(s: AGVSystem, Ft: float) -> float:
    """Productive minutes available from ONE vehicle in one hour."""
    return 60.0 * s.availability * Ft

def estimate_traffic_factor(n_vehicles: float) -> float:
    """
    Simple congestion model when Ft is not supplied.
    A single vehicle has no interference (Ft = 1.0); each additional vehicle on
    the shared guide path costs roughly 5 % of the available time, with a floor
    of 0.60 so the estimate never becomes unphysical.
    """
    return max(0.60, 1.0 - 0.05 * max(0.0, n_vehicles - 1.0))

def fleet_size(s: AGVSystem, max_iter: int = 50) -> dict:
    """
    Compute the required fleet size.
    If traffic_factor is None the calculation is implicit (Ft depends on n and n
    depends on Ft) and is solved by fixed-point iteration.
    """
```

```

"""
cyc = delivery_cycle_time(s)
Tc = cyc["Tc"]
WL = workload(s, Tc)

if s.traffic_factor is not None:
    Ft = s.traffic_factor
    n_exact = WL / available_time(s, Ft)
    iterations = 1
else:
    Ft, n_exact, iterations = 1.0, WL / available_time(s, 1.0), 0
    for iterations in range(1, max_iter + 1):
        Ft_new = estimate_traffic_factor(math.ceil(n_exact))
        n_new = WL / available_time(s, Ft_new)
        if abs(n_new - n_exact) < 1e-6 and abs(Ft_new - Ft) < 1e-9:
            Ft, n_exact = Ft_new, n_new
            break
        Ft, n_exact = Ft_new, n_new

n_req = math.ceil(n_exact - 1e-9)
AT = available_time(s, Ft)
utilisation = n_exact / n_req if n_req else 0.0

return {**cyc, "WL": WL, "Ft": Ft, "AT": AT,
        "n_exact": n_exact, "n_required": n_req,
        "utilisation": utilisation, "iterations": iterations,
        "achievable_rate": n_req * AT / Tc}

# -----
# 3. REPORTING
# -----

def report(s: AGVSystem, r: dict) -> None:
    print("\n" * 80)
    print(f"AGV FLEET SIZING - {s.name}")
    print("\n" * 80)
    print("INPUTS")
    print(f"Loading time TL = {s.load_time:8.3f} min")
    print(f"Unloading time TU = {s.unload_time:8.3f} min")
    print(f"Loaded distance Ld = {s.loaded_distance:8.1f} m")
    print(f"Empty distance Le = {s.empty_distance:8.1f} m")
    print(f"Vehicle speed Vc = {s.velocity:8.1f} m/min")
    print(f"Demand Rf = {s.deliveries_per_hour:8.1f} deliveries/h")
    print(f"Availability A = {s.availability:8.3f}")
    print(f"Traffic factor Ft = ")
    print(f"{'estimated' if s.traffic_factor is None else f'{s.traffic_factor:8.3f}'}")
    print("\n" * 80)
    print("DELIVERY CYCLE TIME")
    print(f"Load = {r['load']:8.3f} min")
    print(f"Travel loaded Ld/Vc = {r['travel loaded']:8.3f} min")
    print(f"Unload = {r['unload']:8.3f} min")
    print(f"Travel empty Le/Vc = {r['travel empty']:8.3f} min")
    print(f"-----")
    print(f"Tc = {r['Tc']:8.3f} min per delivery")
    print("\n" * 80)
    print("FLEET CALCULATION")
    print(f"Workload WL = Rf x Tc = {r['WL']:8.3f} vehicle-min per hour")
    print(f"Traffic factor actually used Ft = {r['Ft']:8.3f}")
    print(f"{'(converged in %d iterations)' % r['iterations'] if s.traffic_factor is None else ''}")
    print(f"Available time AT = 60 x A x Ft = {r['AT']:8.3f} min per vehicle per hour")
    print(f"Vehicles n = WL / AT = {r['n_exact']:8.3f}")
    print(f">>>> REQUIRED FLEET SIZE = {r['n_required']:5d} AGVs (rounded up)")
    print(f"Fleet utilisation = {r['utilisation']*100:8.1f} %")
    print(f"Rate the fleet can actually serve = {r['achievable_rate']:8.2f} deliveries/h")
    print(f"{'demand {s.deliveries_per_hour:.1f}/h' if s.achievable_rate < 1.0 else ''}")
    print("\n" * 80 + "\n")

# -----
# 4. TEST CASES
# -----

def main() -> None:
    print("\n" + "#" * 80)
    print("# TASK 3.4 - AGV FLEET SIZING MODEL : TEST VERIFICATION RUN")
    print("#" * 80 + "\n")

    cases: List[AGVSystem] = []

    # TEST CASE 1 - the FMC loop of Assignment 1 (raw store -> CNC cell)
    cases.append(AGVSystem(
        name="TEST CASE 1 - FMC raw-material loop (Assignment 1 cell)",
        load_time=0.60, unload_time=0.50,
        loaded_distance=140.0, empty_distance=110.0,
        velocity=50.0, deliveries_per_hour=22.0,
        availability=0.95, traffic_factor=0.90))

    # TEST CASE 2 - heavy demand, slow vehicle, congested path
    cases.append(AGVSystem(
        name="TEST CASE 2 - High-demand assembly feed (congested)",
        load_time=1.00, unload_time=0.75,
        loaded_distance=320.0, empty_distance=280.0,
        velocity=40.0, deliveries_per_hour=35.0,
        availability=0.90, traffic_factor=0.85))

    # TEST CASE 3 - traffic factor NOT given, solved implicitly
    cases.append(AGVSystem(
        name="TEST CASE 3 - Traffic factor estimated by iteration",
        load_time=0.75, unload_time=0.75,
        loaded_distance=200.0, empty_distance=150.0,
        velocity=45.0, deliveries_per_hour=40.0,
        availability=0.92, traffic_factor=None))

    results = []
    for s in cases:
        r = fleet_size(s)
        report(s, r)
        results.append((s, r))

    # ---- verification assertions -----
    manual_Tc = (s.load_time + s.loaded_distance / s.velocity
                + s.unload_time + s.empty_distance / s.velocity)
    assert abs(r["Tc"] - manual_Tc) < 1e-9, "cycle time does not match the hand formula"
    assert abs(r["WL"] - s.deliveries_per_hour * manual_Tc) < 1e-9, "workload mismatch"
    assert r["n_required"] >= r["n_exact"] - 1e-9, "fleet size was not rounded UP"
    assert r["n_required"] - r["n_exact"] < 1.0, "rounded up by more than one vehicle"
    assert r["achievable_rate"] >= s.deliveries_per_hour - 1e-6, \
        "the chosen fleet cannot meet the demand"

```

```

    assert 0 < r["utilisation"] <= 1.0 + 1e-9, "utilisation out of range"
    print(" >>> SANITY CHECKS PASSED for this case\n")

# ---- an extra check: demand sweep must give a monotonic fleet size ----
print("=" * 80)
print("MONOTONICITY CHECK - fleet size against rising demand (case 1 geometry)")
print("=" * 80)
base = cases[0]
print(f"Demand, deliveries/h':>22}{ 'n exact':>12}{ 'AGVs required':>16}")
prev = 0
for rf in (5, 10, 20, 30, 40, 60, 80):
    s = AGVSystem(name="sweep", load_time=base.load_time, unload_time=base.unload_time,
                  loaded_distance=base.loaded_distance, empty_distance=base.empty_distance,
                  velocity=base.velocity, deliveries_per_hour=rf,
                  availability=base.availability, traffic_factor=base.traffic_factor)
    rr = fleet_size(s)
    print(f"rf:>22.0f}{rr['n exact']:>12.3f}{rr['n required']:>16d}")
    assert rr["n_required"] >= prev, "fleet size must not fall as demand rises"
    prev = rr["n_required"]
print(" >>> fleet size rises monotonically with demand\n")

print("=" * 80)
print("SUMMARY")
print("=" * 80)
print(f"{'Case':<56}{ 'Tc,min':>9}{ 'AGVs':>7}")
for s, r in results:
    print(f"s.name:<56}{r['Tc']:>9.3f}{r['n_required']:>7d}")
print("=" * 80)
print("ALL TEST CASES COMPLETED SUCCESSFULLY")

if __name__ == "__main__":
    main()

```

```
#!/usr/bin/env python3
"""
Task 3.5 - Automated Data Capture (ADC) Barcode Verification Tool
=====
Parses raw scanned barcode strings, extracts the part serial number, the
operation routing code and the batch data, and validates the scan against a
sample part database so that a component can only be released to the operation
that its routing actually permits.

Two payload formats are supported:

CODE 39    fixed-field, self-checking, framed by the '*' start/stop sentinel
            and terminated by a modulo-43 check character:

            *<PART:8><OP:3><LOT:6><SERIAL:6><CHECK:1>*>
            e.g. *BLKA1234040250715000123K*

QR / 2-D   free-order pipe-delimited key=value payload:

            PN=BLK-A1234|OP=040|LOT=250715|SN=000123

Validation performed
-----
1. sentinel and field-format check
2. Code 39 modulo-43 check character
3. part number exists in the database
4. the operation code belongs to that part's routing
5. the operation is the CORRECT NEXT operation for that serial number
6. the lot code is a real date, not in the future, within shelf life
7. the serial has not already been scanned at this operation (duplicate)

Run: python3 task3_5_barcode_verification.py
"""

from __future__ import annotations

import datetime as _dt
import re
from dataclasses import dataclass, field
from typing import Dict, List, Optional, Tuple

# -----
# 1. CODE 39 CHARACTER SET AND CHECK CHARACTER
# -----
CODE39_CHARSET = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ-.$/+%"
assert len(CODE39_CHARSET) == 43, "the Code 39 character set must hold exactly 43 symbols"

_VALUE = {c: i for i, c in enumerate(CODE39_CHARSET)}

def code39_check_char(data: str) -> str:
    """
    Modulo-43 check character: sum the value of every data character, take the
    remainder modulo 43 and translate it back to a Code 39 symbol.
    """
    total = 0
    for ch in data:
        if ch not in _VALUE:
            raise ValueError(f"character {ch!r} is not in the Code 39 character set")
        total += _VALUE[ch]
    return CODE39_CHARSET[total % 43]

# -----
# 2. PART MASTER DATABASE
# -----

@dataclass
class PartMaster:
    """Routing master record for one part number."""
    part_number: str
    description: str
    routing: List[str] # operation codes in the order they must run
    shelf_life_days: int = 365

    def next_operation(self, completed: List[str]) -> Optional[str]:
        """The operation that is legitimately due next for this part."""
        for op in self.routing:
            if op not in completed:
                return op
        return None

# A small sample database, as required by the task statement.
PART_DB: Dict[str, PartMaster] = {
    "BLK-A1234": PartMaster("BLK-A1234", "Engine cylinder block, 4-cyl",
                           ["010", "020", "030", "040", "050", "060"]),
    "SHF-T5500": PartMaster("SHF-T5500", "Transmission stepped shaft",
                           ["010", "020", "040", "070"]),
    "PIN-L0099": PartMaster("PIN-L0099", "Aluminium locating pin",
                           ["010", "030", "070"], shelf_life_days=180),
}

OPERATION_NAMES = {
    "010": "Raw material receipt / goods-in inspection",
    "020": "CNC rough turning",
    "030": "CNC milling",
    "040": "CNC finish turning / threading",
    "050": "Deburr and wash",
    "060": "In-line dimensional inspection",
    "070": "Final inspection and despatch",
}

# -----
# 3. PARSED SCAN RECORD
# -----

@dataclass
class ScanRecord:
    """A decoded barcode."""
    raw: str
    symbology: str
    part_number: str = ""

```

```

operation: str = ""
lot: str = ""
serial: str = ""
check_char: str = ""
parse_errors: List[str] = field(default_factory=list)

@property
def ok(self) -> bool:
    return not self.parse_errors

@property
def unique_id(self) -> str:
    return f"{self.part_number}/{self.lot}/{self.serial}"

# -----
# 4. PARSERS
# -----

# *<PART:8><OP:3><LOT:6><SERIAL:6><CHECK:1>*
_C39_BODY = re.compile(r"^\*([0-9A-Z\-. $/+] {23})([0-9A-Z\-. $/+] )\*$")

def parse_code39(raw: str) -> ScanRecord:
    """Decode a fixed-field Code 39 payload and verify its check character."""
    rec = ScanRecord(raw=raw, symbology="CODE39")
    s = raw.strip().upper()

    m = _C39_BODY.match(s)
    if not m:
        if not (s.startswith("*") and s.endswith("*")):
            rec.parse_errors.append("missing '*' start/stop sentinel")
        else:
            rec.parse_errors.append(
                f"wrong payload length: expected 24 data+check characters, got {len(s) - 2}")
        return rec

    body, check = m.group(1), m.group(2)
    rec.check_char = check

    expected = code39_check_char(body)
    if check != expected:
        rec.parse_errors.append(
            f"modulo-43 check character is {check!r}, expected {expected!r}")

    # fixed-width fields - the part number is stored without its hyphens
    packed_pn = body[0:8].strip()
    rec.operation = body[8:11]
    rec.lot = body[11:17]
    rec.serial = body[17:23]

    # restore the hyphenated master part number (BLKA1234 -> BLK-A1234)
    rec.part_number = _unpack_part_number(packed_pn)

    if not rec.operation.isdigit():
        rec.parse_errors.append(f"operation field {rec.operation!r} is not numeric")
    if not rec.serial.isdigit():
        rec.parse_errors.append(f"serial field {rec.serial!r} is not numeric")
    return rec

def _unpack_part_number(packed: str) -> str:
    """Match a de-hyphenated scan against the hyphenated master part numbers."""
    for pn in PART_DB:
        if pn.replace("-", "") == packed:
            return pn
    return packed # unknown - reported later by the validator

_QR_FIELD = re.compile(r"^[A-Z]{2,3}=(.+)$")

def parse_qr(raw: str) -> ScanRecord:
    """Decode a pipe-delimited key=value QR payload."""
    rec = ScanRecord(raw=raw, symbology="QR")
    mapping: Dict[str, str] = {}

    for token in raw.strip().split("|"):
        token = token.strip()
        if not token:
            continue
        m = _QR_FIELD.match(token.upper())
        if not m:
            rec.parse_errors.append(f"malformed field {token!r} (expected KEY=VALUE)")
            continue
        mapping[m.group(1)] = m.group(2)

    for key, attr in (("PN", "part number"), ("OP", "operation"),
                     ("LOT", "lot"), ("SN", "serial")):
        if key not in mapping:
            rec.parse_errors.append(f"mandatory field {key} is missing")
        else:
            setattr(rec, attr, mapping[key])
    return rec

def parse_scan(raw: str) -> ScanRecord:
    """Detect the symbology automatically and dispatch to the right parser."""
    s = raw.strip()
    if s.startswith("*"):
        return parse_code39(s)
    if "=" in s:
        return parse_qr(s)
    rec = ScanRecord(raw=raw, symbology="UNKNOWN")
    rec.parse_errors.append("payload matches neither the Code 39 nor the QR format")
    return rec

# -----
# 5. ROUTING VALIDATION
# -----

class RoutingValidator:
    """Checks a decoded scan against the part master and the scan history."""
    def __init__(self, db: Dict[str, PartMaster], today: _dt.date) -> None:
        self.db = db
        self.today = today
        # history[unique_id] = list of operation codes already completed

```

```

self.history: Dict[str, List[str]] = {}

# ---- individual checks -----
def _check_lot(self, rec: ScanRecord, master: PartMaster) -> List[str]:
    errs: List[str] = []
    if not re.fullmatch(r"\d{6}", rec.lot):
        return [f"lot code {rec.lot!r} is not a 6-digit YYMMDD date"]
    try:
        lot_date = dt.datetime.strptime(rec.lot, "%y%m%d").date()
    except ValueError:
        return [f"lot code {rec.lot!r} is not a valid calendar date"]
    if lot_date > self.today:
        errs.append(f"lot date {lot_date} lies in the future")
    age = (self.today - lot_date).days
    if age > master.shelf_life_days:
        errs.append(f"lot is {age} days old, shelf life is {master.shelf_life_days} days")
    return errs

# ---- the public entry point -----
def validate(self, rec: ScanRecord) -> Tuple[bool, List[str], List[str]]:
    """Return (accepted, errors, notes)."""
    errors = list(rec.parse_errors)
    notes: List[str] = []

    if errors:
        return False, errors, notes

    master = self.db.get(rec.part_number)
    if master is None:
        errors.append(f"part number {rec.part_number!r} is not in the part master")
        return False, errors, notes
    notes.append(f"part recognised: {master.description}")

    errors.extend(self._check_lot(rec, master))

    if rec.operation not in master.routing:
        errors.append(f"operation {rec.operation} is not in the routing for "
                      f"{rec.part_number} (routing: {'-'.join(master.routing)})")
    else:
        notes.append(f"operation {rec.operation} = "
                     f"OPERATION_NAMES.get(rec.operation, 'unnamed')")
        done = self.history.get(rec.unique_id, [])
        if rec.operation in done:
            errors.append(f"duplicate scan: operation {rec.operation} has already "
                          f"been recorded for serial {rec.serial}")
        else:
            expected = master.next_operation(done)
            if expected != rec.operation:
                errors.append(f"routing violation: operation {rec.operation} presented "
                              f"but operation {expected} is the next one due "
                              f"(completed so far: {'-'.join(done)} or 'none')")
            else:
                notes.append(f"routing position correct "
                             f"({len(done)+1} of {len(master.routing)})")

    accepted = not errors
    if accepted:
        self.history.setdefault(rec.unique_id, []).append(rec.operation)
    return accepted, errors, notes

# -----
# 6. REPORTING
# -----

def process(raw: str, validator: RoutingValidator, label: str = "") -> bool:
    rec = parse_scan(raw)
    accepted, errors, notes = validator.validate(rec)

    print("-" * 78)
    if label:
        print(f" {label}")
    print(f" RAW SCAN : {raw}")
    print(f" SYMBOLOLOGY : {rec.symbology}")
    if rec.part_number or rec.operation or rec.serial:
        print(f" DECODED : part={rec.part_number or '?'} op={rec.operation or '?'} "
              f"lot={rec.lot or '?'} serial={rec.serial or '?'} "
              f"+ (f\"check={rec.check_char}\" if rec.check_char else \"\")")
    for n in notes:
        print(f" note : {n}")
    for e in errors:
        print(f" ERROR : {e}")
    print(f" RESULT : {'ACCEPT - part may proceed' if accepted else 'REJECT - part diverted to the query bay'}")
    return accepted

def build_code39(part: str, op: str, lot: str, serial: str,
                 corrupt_check: bool = False) -> str:
    """Helper that encodes a valid (or deliberately corrupted) Code 39 payload."""
    body = f"{part.replace('-', '').<8}{op:>03}{lot}{serial:>06}"
    if len(body) != 23:
        raise ValueError(f"body must be 23 characters, got {len(body)}: {body!r}")
    chk = code39_check_char(body)
    if corrupt_check:
        chk = CODE39_CHARSET[(CODE39_CHARSET.index(chk) + 1) % 43]
    return f"*{body}{chk}*"

# -----
# 7. TEST CASES
# -----

def main() -> None:
    print("\n" + "-" * 78)
    print("# TASK 3.5 - ADC BARCODE VERIFICATION TOOL : TEST VERIFICATION RUN")
    print("#" * 78)

    # A fixed 'today' keeps the test output reproducible.
    TODAY = dt.date(2026, 3, 1)
    v = RoutingValidator(PART_DB, TODAY)
    print(f"\n Validation date fixed at {TODAY} for reproducibility")
    print(f" Part master holds: {'-'.join(PART_DB)}\n")

    # --- check-character self test -----
    print("-" * 78)
    print("CHECK-CHARACTER SELF TEST (modulo 43)")
    print("-" * 78)
    for data in ("BLKA1234", "SHFT5500010", "123456"):
        print(f" data {data:<14} -> check character {code39_check_char(data)!r}")

```

```

assert code39_check_char("") == "0", "empty data must give check character '0'"
print("    >>>-check-character routine behaves as specified\n")

results = []

# ===== TEST CASE 1 - a clean Code 39 scan at the correct operation =====
print("=" * 78)
print("TEST CASE 1 - valid Code 39 scan, correct first operation")
print("=" * 78)
sl = build_code39("BLK-A1234", "010", "260115", "000123")
r = process(sl, v, "goods-in scan of an engine block")
assert r is True, "a clean, in-sequence scan must be accepted"
results.append(("valid Code 39, op 010", r, True))

# the next operation in the routing must now be accepted too
slb = build_code39("BLK-A1234", "020", "260115", "000123")
r = process(slb, v, "same serial presented at the next operation")
assert r is True, "the next routing operation must be accepted"
results.append(("valid Code 39, op 020 (next)", r, True))

# ===== TEST CASE 2 - QR payload, routing violation =====
print("\n" + "=" * 78)
print("TEST CASE 2 - QR payload with a routing violation and a duplicate")
print("=" * 78)
r = process("PN=BLK-A1234|OP=060|LOT=260115|SN=000123", v,
            "operation 060 presented while 030 is due")
assert r is False, "an out-of-sequence operation must be rejected"
results.append(("QR, out-of-sequence op", r, False))

r = process("PN=BLK-A1234|OP=020|LOT=260115|SN=000123", v,
            "operation 020 presented a second time")
assert r is False, "a duplicate scan must be rejected"
results.append(("QR, duplicate scan", r, False))

r = process("PN=SHF-T5500|OP=010|LOT=260210|SN=004501", v,
            "a different part, first operation")
assert r is True, "a valid scan of another part must be accepted"
results.append(("QR, valid other part", r, True))

# ===== TEST CASE 3 - corrupted and unknown scans =====
print("\n" + "=" * 78)
print("TEST CASE 3 - corrupted check character, unknown part, bad lot, junk")
print("=" * 78)
r = process(build_code39("BLK-A1234", "030", "260115", "000123", corrupt_check=True), v,
            "one bit flipped in the check character")
assert r is False, "a bad modulo-43 check character must be rejected"
results.append(("Code 39, bad check char", r, False))

r = process("PN=XXX-99999|OP=010|LOT=260115|SN=000001", v, "part not in the master")
assert r is False, "an unknown part number must be rejected"
results.append(("QR, unknown part", r, False))

r = process("PN=PIN-L0099|OP=010|LOT=270101|SN=000045", v, "lot dated in the future")
assert r is False, "a future lot date must be rejected"
results.append(("QR, future lot date", r, False))

r = process("PN=PIN-L0099|OP=010|LOT=240101|SN=000046", v,
            "lot older than the 180-day shelf life")
assert r is False, "an expired lot must be rejected"
results.append(("QR, expired lot", r, False))

r = process("HELLO WORLD 12345", v, "unreadable payload")
assert r is False, "unparseable data must be rejected"
results.append(("junk payload", r, False))

r = process("SHORT*", v, "truncated Code 39 payload")
assert r is False, "a truncated payload must be rejected"
results.append(("Code 39, truncated", r, False))

# ===== summary =====
print("\n" + "=" * 78)
print("SUMMARY OF ALL SCANS")
print("=" * 78)
print(f"{'Scan':<34}{'Result':>10}{'Expected':>12}{'Verdict':>12}")
allok = True
for name, got, want in results:
    ok = got == want
    allok &= ok
    print(f"{name:<34}{'ACCEPT' if got else 'REJECT':>10}"
          f"{'ACCEPT' if want else 'REJECT':>12}{'OK' if ok else 'MISMATCH':>12}")
print("-" * 78)
print(f"Scans processed : {len(results)}")
print(f"Accepted       : {sum(1 for _, g, _ in results if g)}")
print(f"Rejected        : {sum(1 for _, g, _ in results if not g)}")
assert allok, "at least one scan did not give the expected verdict"
print("\n Routing history recorded by the validator:")
for uid, ops in v.history.items():
    print(f"    {uid:<28} operations completed: {'-'.join(ops)}")
print("=" * 78)
print("ALL TEST CASES COMPLETED SUCCESSFULLY")

if __name__ == "__main__":
    main()

```